

## AFDPS for Linear Inverse Scattering -- Setup & Run Guide

This guide takes you from a fresh checkout to the full InverseBench Table-3 comparison for the AFDPS linear-inverse-scattering port: applying AFDPS (Approximation-Free Diffusion Posterior Sampling, TMLR 2026) to recover the permittivity contrast from a complex scattered light field. The numerical core is already built and VERIFIED on CPU (24/24 tests); the remaining work is asset download and the GPU (GB200) runs.

All AFDPS-scattering code lives under the `inverse_scattering/` directory. The InverseBench harness, forward operator, dataset loader, EDM prior, and evaluator are REUSED by import from the sibling `navier_stokes/` tree, untouched, so observation generation and metrics are exactly the benchmark's and the comparison to Table 3 is fair.

Goal: place the AFDPS PSNR / SSIM / Meas-err next to the InverseBench Table 3 baselines at 360 / 180 / 60 receivers (noise  $\sigma_y = 1e-4$ ). Best PnP-diffusion baselines @360 receivers: RED-diff 36.56, DDNM 36.38 PSNR; DiffPIR / PnP-DM 0.988 SSIM.

---

### Why scattering is different from (and easier than) Navier-Stokes

The Navier-Stokes port had a NONLINEAR PDE forward, so its likelihood gradient needed a discrete-adjoint solve and its log-likelihood Laplacian needed a stochastic Hutchinson estimate. Linear inverse scattering (first Born approximation) has a LINEAR forward operator  $A$ , and the benchmark operator already caches a real SVD  $A = U S V^T$ . So the entire AFDPS likelihood engine here is exact and closed-form:

- gradient:  $\text{grad } \mu_y = A^T (A x - y) / \sigma^2$  (one  $V/U$  projection, no autograd)
- Laplacian:  $\text{Tr}(\text{Hessian}) = \sum_i s_i^2 / \sigma^2$  (an exact constant, no Hutchinson)
- PiGDM:  $A^T (\sigma^2 I + \sigma^4 A A^T)^{-1} r$  (diagonal in the  $U$ -basis, no CG)
- guidance: the linear guidance ODE is integrated EXACTLY each step in the  $V$ -basis (unconditionally stable; a DDNM-like data projection at the final step).

Two consequences shape the defaults. (1)  $\sigma_y = 1e-4$  is near-noiseless, so the problem is conditioning-dominated: enforce data on well-measured singular directions, keep the prior on the null space. (2) Table 3 ranks POINT estimates (PSNR/SSIM), whose leaders are effectively posterior-mean estimators -- so the reported AFDPS estimate is the weighted ENSEMBLE MEAN, not a single posterior sample.

---

### Where things stand (what is already done)

- Operator, sampler, algorithm, configs, Hydra harness, GB200 scripts, and an aggregator are all implemented under `inverse_scattering/`.
- Numerical core VERIFIED on CPU (24/24 tests, ~3 s): the analytic gradient matches autograd through the UNTOUCHED benchmark loss; the exact Laplacian and Jacobian trace match; closed-form PiGDM matches a dense solve; the exact-linear guidance substep matches a 20k-step Euler integration; sampler determinism; fairness guards (no operator overrides); Hydra composition + searchpath; and the linear-Gaussian TWIN -- the AFDPS weighted mean recovers the closed-form posterior mean on the measured subspace.
- The forward operator, dataset, EDM-prior architecture, and evaluator are the benchmark's, reused by import (fairness).

So the next steps are purely: download assets, then run on the GPU box (smoke -> validation sweep -> full test -> aggregate).

---

### Prerequisites

- A Linux GPU box (the GB200). NVIDIA driver + CUDA runtime matching your torch build.

- Python 3.9+ (3.10 / 3.11 recommended).
  - ~5-10 GB disk for the checkpoint + datasets + SVD caches. Internet to GitHub + Caltech data.
  - Both trees side by side: `inverse_scattering/` and its sibling `navier_stokes/` (the harness is imported from the latter).
- 

## Step 1 -- Get the repo and enter the scattering directory

Clone the repository and `cd` into the `inverse_scattering` directory. Every command below is run from there (the entry points pin the working directory so `../data`, `checkpoints/`, `cache/`, `exps/` and the Hydra searchpath `../navier_stokes/configs` all resolve consistently).

```
git clone https://github.com/erichou1/AFDPS.git
cd AFDPS/inverse_scattering
```

Confirm the sibling harness is present (the AFDPS-scattering code imports it):

```
ls ../navier_stokes/main.py ../navier_stokes/eval.py # should exist
```

---

## Step 2 -- Environment

Create a virtual environment and install the dependencies. On a box that already has a system torch, `--system-site-packages` lets you reuse it:

```
python -m venv --system-site-packages .venv
source .venv/bin/activate
pip install torch numpy scipy pyyaml tqdm omegaconf hydra-core lmdb piq requests fpdf
```

Inference and verification need neither wandb nor an account.

---

## Step 3 -- CPU verification (no GPU, no checkpoint, no data)

The whole value of the SVD likelihood engine is that it is correct. Run the ladder; it finishes in seconds:

```
python -m pytest tests/ -q # expect: 24 passed
```

This proves the gradient / Laplacian / PiGDM / exact-linear integrator / affine fold / Feynman-Kac weighting are all correct against the untouched benchmark, BEFORE any GPU time.

---

## Step 4 -- Download assets (checkpoint + data)

Download the pretrained diffusion prior and the inverse-scattering test (and validation) datasets. Run on the box where you will infer:

```
bash scripts/download_assets.sh # checkpoint + test/val data
# bash scripts/download_assets.sh ckpt # checkpoint only
# bash scripts/download_assets.sh data # data only
```

This writes `checkpoints/inv-scatter-5m.pt` and `../data/inv-scatter-{test,val}`. The script tries both known release-asset names (`inv-scatter-5m.pt` and `in-scatter-5m.pt`). If the data download 401/403s, the Caltech share token has expired -- refresh it in the script header (see the Troubleshooting section).

---

## Step 5 -- Precompute the SVD caches (once, up front)

The forward operator builds a cached SVD the first time it is constructed (10-20 min) and writes it under `cache/`. Build all three receiver counts serially BEFORE any parallel run so sharded jobs never race on the build:

```
python scripts/precompute_svd.py --numTrans 20 --numRec 360 180 60
```

A cache is treated as complete only when `matrix_inv.pt` exists (the last file written), so a re-run safely skips

finished caches and rebuilds partial ones.

---

## Step 6 -- GB200 smoke + throughput benchmark

Validate the GPU path and pick the batch size that fills the device. This asserts the real checkpoint exposes the EDM-net interface FIRST, runs one real inference case, then benchmarks per-step time and peak memory at J in {512, 1024, 2048}:

```
bash scripts/smoke_gb200.sh
```

Pick the largest particle count J that fits and keeps utilization high. Watch utilization live in another shell with the GPU logger:

```
bash scripts/gpu_log.sh start exps/util.csv
# ... run something ...
bash scripts/gpu_log.sh stop exps/util.csv
bash scripts/gpu_log.sh summarize exps/util.csv # mean / median / p10 util, peak mem
```

---

## Step 7 -- Validation sweep (find the best AFDPS config per receiver count)

Run a small coordinate-descent grid on the 10 validation cases and rank by PSNR. The grid covers the guidance family (full anisotropic PiGDM / auto isotropic / fixed-gamma), the guidance step (exact\_linear vs the paper-faithful euler), step count, reduction (mean vs best), and resampling:

```
bash scripts/run_val_sweep.sh 360 512 # receiver-count J
bash scripts/run_val_sweep.sh 180 512
bash scripts/run_val_sweep.sh 60 512
```

It prints a ranked table (scripts/val\_table.py). Use the winning config for that receiver count in the full test run. The promoted primary is full PiGDM + exact\_linear + ensemble mean; expect it to lead at 360 / 180, with more particles / resampling helping most at 60.

---

## Step 8 -- Full test set + Table-3 comparison

Run all 100 test cases for each receiver count, sharded across co-tenant GPU processes with utilization logging, then aggregate into the Table-3 format:

```
bash scripts/run_test_all.sh 2 1024 200 # NSHARD J STEPS (360 -> 180 -> 60)
```

Or one receiver count at a time, then aggregate manually:

```
bash scripts/run_test.sh 360 2 1024 200
python scripts/aggregate_table3.py --numRec 360 \
    "exps/inference/inverse-scatter-afdps/AFDPS/final_R360_shard*/result_*.pt"
```

The aggregator prints the full 12-method InverseBench baseline table for the chosen receiver count with the AFDPS row appended (markdown + LaTeX), and flags whether AFDPS leads on PSNR.

A single reference inference run (no sharding):

```
python main.py problem=inv-scatter-afdps algorithm=afdps pretrain=inv-scatter \
    num_samples=1 wandb=false problem.model.numRec=360
```

---

## Tuning knobs (override on the CLI)

- algorithm.method.num\_particles -- ensemble size; raise to fill the GPU (batched).
- algorithm.method.num\_steps -- annealed-SDE steps (100-400).
- algorithm.method.sampler\_kwargs.guidance\_mode -- full (anisotropic PiGDM, primary) | auto (isotropic,  $\gamma_e^2 = \lambda_{\bar{}}$ ) | fixed (hand-set guidance\_gamma).

- `algorithm.method.sampler_kwargs.guidance_step -- exact_linear (primary) | euler (paper-faithful AFDPS-SDE ablation).`
- `algorithm.method.reduce -- mean (Table-3 number) | best / topk (posterior-sample draws).`
- `algorithm.method.guidance_gamma -- only used by guidance_mode=fixed.`

Paper-faithful AFDPS-SDE ablation example:

```
python main.py problem=inv-scatter-afmps algorithm=afmps pretrain=inv-scatter wandb=false \
  algorithm.method.sampler_kwargs.guidance_mode=fixed \
  algorithm.method.sampler_kwargs.guidance_step=euler \
  algorithm.method.guidance_gamma=10 algorithm.method.reduce=best
```

## GB200 efficiency

- The ensemble is batched across particles (one UNet forward over J particles per step) -- raise `num_particles` to saturate the device; `smoke_gb200.sh` benchmarks J to pick the largest that fits.
- Cases are seeded per global id before observation generation, so the 100-case test set can be sharded across co-tenant processes (`run_test.sh` NSHARD) with the aggregated mean PSNR independent of shard count.
- `gpu_log.sh` reports mean / median / p10 utilization so you can confirm the device stays near 100%. Benchmark single-shard utilization first; only raise NSHARD if a real gap remains (each shard reloads the multi-GB SVD factors and contends for SMs).
- Per-case seeding reproduces INPUTS, not bitwise outputs (CUDA GEMM/conv nondeterminism), unless `torch.use_deterministic_algorithms(True)` -- the realistic guarantee is statistically identical mean PSNR.

## Troubleshooting

- Data download 401/403: the Caltech share token expired -> refresh `CALTECH_TOKEN` in `scripts/download_assets.sh` (get a fresh link from the InverseBench data record).
- Checkpoint name: the release asset has been seen as both `inv-scatter-5m.pt` and `in-scatter-5m.pt`; the downloader tries both. If neither resolves, update `CKPT_CANDIDATES`.
- Parallel shards race / load errors on cache: run `scripts/precompute_svd.py` FIRST so every SVD cache is complete before launching shards.
- CUDA out of memory: lower `algorithm.method.num_particles` (and/or NSHARD). The SVD factors at `R=360` add a few GB on top of the ensemble.
- PSNR below the baselines: confirm `reduce=mean` (not best), prefer `guidance_mode=full` + `guidance_step=exact_linear`, and re-check the per-receiver winner from the validation sweep.

## Open items to confirm before publishing

- A separate `inv-scatter-val` LMDB (10 images) is downloadable; otherwise split the test set.
- The release checkpoint name and that the Caltech share token is still valid.
- `numTrans = 20` matches the InverseBench Table-3 setup.
- The Meas-err (%) definition: the aggregator uses  $100 * ||A \hat{x} - y|| / ||y||$ ; pin this to the InverseBench appendix before reporting the comparison column.

## Key files & reference

- `README.md` -- overview + the same run commands.
- `plan.md` -- full design, derivations (affine fold,  $\sqrt{2}$  noise, exact-linear), risks.
- `inverse_problems/inverse_scatter_afmps.py` -- AFDPSInverseScatter (exact SVD engine).
- `algo/afmps_scatter.py` + `algo/afmps_core_scatter/ensemble_denoiser_edm.py` -- algorithm + sampler.

- configs/{config.yaml, problem/inv-scatter-afdps.yaml, algorithm/afdps.yaml}.
- scripts/ -- download\_assets.sh, precompute\_svd.py, smoke\_gb200.sh, run\_val\_sweep.sh, run\_test.sh, run\_test\_all.sh, gpu\_log.sh, aggregate\_table3.py, val\_table.py, metrics\_lib.py.
- tests/ -- the 24-check CPU verification ladder (operator, configs, sampler twin, aggregator).